后训练学习计划 • 第 12 周（11/23-11/29）

本周定位：**Phase 3 / M4 收官——评测与集成**。通用基准（带 base 对照，专门查对齐有没有把通用能力训坏）、自建 200 题教学评测集 + LLM-as-judge、在真实"明日记忆"里用 vLLM 接上小模型做开关式灰度 A/B、模型上传 HF/ModelScope、可复现 repo 收口、M4 系列博客第 3 篇、面试叙事 200 字定稿。 **Day 7 是 Phase 3 的总验收，也是整个技术能力部分（周 1-12）的终点**，验收判据比前 11 周都硬：能跑的以能跑为准，不能跑的当场补，补不上的记进周 13-16。

用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗红字 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	通用基准（lm-eval-harness，4 个模型带 base 对照）	`eval/baseline/` 结果表 + 退化归因结论	6h
Day 2 周二	自建 200 题教学评测集 + LLM-as-judge 协议	`teach200.jsonl` + `rubric.md` + 10 题冒烟 + 花费测算	6h
Day 3 周三	vLLM 起服务 + 明日记忆接入开关（off/shadow/on）	`vllm serve` 可访问 + 单测全绿 + 影子双跑 30 条	6.5h
Day 4 周四	30 场景 A/B 收口 + 成本/延迟对比表	`ab_report.md` 三列对比表 + 失败请求归档	6h
Day 5 周五	模型上传 HF/ModelScope + repo 可复现收口	两个可访问的模型链接 + 从零 clone 跑通	6.5h
Day 6 周六	M4 系列博客第 3 篇 + 面试叙事 200 字定稿 + 讲稿	博客已发布 + 叙事定稿 + 15 分钟讲稿	6h
Day 7 周日	补漏 + 闭卷复测 + **Phase 3 总验收** + 周 13 预习	9 项验收 ≥8 通过且 3 项强制项全过	5h

注：**本周卡时上限 10h，预估花费 ≈¥20-26（4090）+ LLM-as-judge API ≈¥5**，合计 **≤¥30**。分配：Day 1 评测 4h（4 个模型 × 4 个任务，带 `--limit 300`）、Day 3 起服务与压测 2.5h、Day 4 补跑 1h、Day 5 上传前最终校验 1.5h、Day 6 博客补图 1h。**超了怎么办**：先砍 Day 6 的补图（用 Day 1/Day 4 已有截图），再砍 Day 1 的第 4 个任务（`mmlu_high_school_computer_science`），**Day 3 的 2.5h 不许砍**——服务起不来，本周主线（真实接入 A/B）直接归零。

注：**本周最大风险是"三处都对不上数"**：（1）评测口径不一致（base 与后训练模型用了不同 `--limit` /温度，差值无意义）；（2）judge 不稳定（同一份输出三次打分差 2 分，评测集白建）；（3）A/B 数字与简历数字对不上（今天不锁死溯源表，周 13 写简历时你就会开始凭印象编数）。**砍单顺序**：Day 4 任务 3 的主观分补测 → Day 1 的 OpenCompass 交叉校验（选做项）→ Day 5 的 ModelScope（延到周 13 Day 1，**不许延到投递之后**）→ 其余不许砍。

注：本周**不新增任何训练**。所有需要权重的动作只有两件事：跑评测、起推理服务。如果发现自己在改 LoRA rank、想再跑一轮 DPO——停下来，那是周 11 的活，本周的产出物是"可信的读数 + 能跑的集成"。

二、开始前：环境 / 素材 / 上卡准备（约 40 分钟，前一晚做完）

1. **本地目录与依赖**：

```
mkdir -p E:/study/posttrain/week12/{day1,day2,day3,day4,day5,day6}
cd E:/study/posttrain/llm-posttraining-lab && git pull
source E:/study/posttrain/.venv/Scripts/activate
pip config set global.index-url https://pypi.tuna.tsinghua.edu.cn/simple
pip install jsonschema pytest pandas openai -q
```

2. **周 11 交接物核对**（缺任何一项，Day 1 前 2 小时先补，这是周 11 Day 7 写进 `## 交给周 12` 的清单）：
   - `m4-capstone/align/`：`sft-merged` / `dpo-merged` / `grpo-merged`（或降级说明）、`RESULTS.md`、`FAILURES.md`（≥6 条）、Day 5 的 100 条 held-out prompt 清单。
   - `m4-capstone/ab-protocol.md`：30 个真实查词场景是否已冻结成 `ab_scenarios_30.jsonl`。
   - 周 7 的 `english-tutor-instruct` 数据集页 + data card 链接（写 model card 要引它）。
   - 周 11 被砍掉的**退化快测**（Day 1 的正式基准要覆盖它）。
3. **服务端仓库位置确认**（明天要改代码，别到那天才找文件）：
   - `packages/agent/src/tools/llm-router.ts`：`LlmRouter` 的角色是 `'strong' | 'fast' | 'classify'`，已有 `fallbackBaseUrl / fallbackApiKey / fallbackModel` 三个可选参数——**这就是我们的接入点，不用改架构**。
   - `packages/agent/src/nodes/tutor.ts`：`TutorContentSchema` + `router.createStructured('strong', TutorContentSchema)`。
   - `packages/server/src/routes/tutor.ts`：`GET /api/tutor?word=...&level=B1` → `{word, level, content, cached}`。
4. **开卡准备**：把周 11 的 `start_w11.sh` 复制为 `start_w12.sh`（改日志前缀），确认里面有 `export HF_ENDPOINT=https://hf-mirror.com`。

> 注：`source /etc/network_turbo` 对 github/hf/pypi 有效、对 wandb 常无效；本周评测走 `lm-eval` 要下数据集（GSM8K/HellaSwag/ARC），**huggingface 域名走镜像，但有些任务的 dataset 脚本在 GitHub 上**——两个都要加速。学术加速完全不可用时，改从 ModelScope 下数据集再本地指路径（`modelscope download --dataset ...`），别硬等。

Day 1 • 周一：通用基准 + base 对照（防对齐把通用能力训坏）

今日总览（6h，卡时 4h）：四个模型（原始 base `Qwen2.5-1.5B`、`sft-merged`、`dpo-merged`、`grpo-merged`）在**同一份评测配置**下跑 4 个任务，出一张退化表。今天的关键不是分数好看，是**口径完全一致**——不一致的差值没有任何解释力。

任务 1：算账 + 装评测环境（约 40 分钟）

1. `COST.md` 写 Day 1 行：**4h × ¥1.5-2.5 ≈ ¥6-10**，砍单条件：跑不完就减任务不减模型。

2. 开机后装：

```
source /etc/network_turbo || true
export HF_ENDPOINT=https://hf-mirror.com
pip install "lm-eval[models,samples]@https://github.com/EleutherAI/lm-evaluation-
harness/archive/refs/heads/main.zip" -q \
  || pip install lm-eval -q
lm-eval --help | head -20
python -c "import lm_eval; print(lm_eval.__version__)"
```

完成判据：`lm-eval --help` 打得出来（或旧版 `python -m lm_eval --help`）；版本号记进 `requirements-w12.txt`。**安装超过 30 分钟就换方案**：直接用 OpenCompass（`pip install opencompass`）跑同类任务，或退化为"自己写 200 题选择题的 exact-match 评测"（记进 gaps，别耗一整天）。

> 注：CLI 名字在版本间变过（老版 `lm_eval`、新版 `lm-eval`），装哪个用哪个；`--tasks` 的名字也要用 `lm-eval tasks --print`（或 `python -m lm_eval --tasks list`）现场查，**不要照抄任何文档里的任务名**——任务名改了会静默跳过某些任务，你会拿到一张少一列的表还以为跑完了。这是今天最阴的一个坑。

任务 2：定配置并写死（约 50 分钟）

评测配置一次定死，四个模型共用，存 `m4-capstone/eval/eval_config.md`：

项	取值	为什么
任务	`gsm8k`（数学、生成式）、`hellaswag`（常识）、`arc_easy`（推理）、`mmlu_high_school_computer_science`（知识）	覆盖四类，1.5B 都能在 4090 上跑得动
`--limit`	300（每个任务取前 300 题）	全量 hellaswag 一万题太贵；**四个模型必须同一个 limit**
精度	bf16	与训练一致
生成参数	`temperature=0.0, max_gen_toks=256`	GSM8K 是生成式任务，必须写死
`--seed`	42	抽样与 few-shot 选择都受影响
`repeats`	1	时间预算不够做多次，改在 Day 2 的一致性抽查上花功夫

完成判据：`eval_config.md` 六个字段齐；并能一句话回答"为什么 GSM8K 要写 `temperature=0`"（答不上去周 5 的采样对照笔记）。

> 注：`--limit` 是"截断到前 N 题"，不是随机抽样。截断集与全集会有系统性偏差（GSM8K 前 300 题偏易），所以**这张表只能用于同一份截断下的纵向对比（base → SFT → DPO → GRPO），不能报"绝对正确率"**。简历与博客里要写"在 GSM8K 前 300 题（同一截断）上，从 x% 到 y%"，不许写"模型 GSM8K 85%"。这是数据诚实，也是面试官很爱戳的地方。

任务 3：跑四个模型（约 180 分钟）

```
cd /root/autodl-tmp/m4
for M in base sft-merged dpo-merged grpo-merged; do
  P=/root/autodl-tmp/m4/$M
  [ "$M" = base ] && P=Qwen/Qwen2.5-1.5B
  lm-eval --model hf --model_args "pretrained=$P,dtype=bfloat16" \
    --tasks gsm8k,hellaswag,arc_easy,mmlu_high_school_computer_science \
    --batch_size 8 --limit 300 --seed 42 \
    --gen_kwargs "temperature=0.0,max_gen_toks=256" \
    --output_path "eval/$M" 2>&1 | tee "logs/eval_${M}_$(date +%m%d_%H%M).log"
done
```

`base` 那次如果报 `use_chat_template` 相关错，加 `--model_args "pretrained=$P,dtype=bfloat16,use_chat_template=False"`（base 没有对话模板，**四个模型里只有 base 需要这一句**，写进 `eval_config.md` 的"差异说明"，别偷偷不一致）。

完成判据：`eval/` 下四个目录各有一个 `results_*.json`；**每个模型的四列里，任务列名一致、题数一致**（`--limit` 生效）；无模型缺跑。

> 注：`--batch_size auto` 在生成式任务上会报 `ValueError: 'auto' batch size is not supported for this task`（预期现象，不是你的错），改成固定 `8`；若 bf16 下 GSM8K 出现大量 `NaN`，先怀疑 `dtype` 写错或 model_args 里逗号后有空格被解析进路径。**单模型跑超 50 分钟就 `Ctrl+C`，把 limit 降到 200 重跑四个**（一致性优先于题量，四个都用 200）。

任务 4：退化表 + 归因（约 60 分钟）

`m4-capstone/eval/DEGRADATION.md`：四行（模型）× 四列（任务）+ 相对 base 的差值列。然后做归因，三条必答： 1. 哪一档掉得最多？掉在生成式任务还是选择题任务上？ 2. 如果 SFT 就掉：数据是否太窄（周 7 的教学对话几乎不含数学/常识）→ 结论是**必须混 replay**（通用指令混入 5-10%）。 3. 如果只有 DPO/GRPO 掉：是 KL 跑偏还是格式收紧挤掉了知识表达 → 引周 11 的 KL 曲线数字。

完成判据：表格完整 + **最大退化值写清楚**（"最大退化 -X.X pt，出现在 任务 / 档位"）+ 三条归因各 ≥3 行且引用具体文件。**若退化 >10 pt → 立刻在 gaps 立一条"P3 必须加 replay 重跑"，并在 Day 6 的面试叙事里把它写成已知局限，不许藏。**

> 注：面试官真正会问的是"你怎么知道你的对齐没把模型训坏"。所以**这张表的存在比表里的数字重要**：它证明你知道要设 base 对照、知道要用同一份截断、知道生成式任务的解码参数会污染结论。

任务 5：关机 + 记账（约 30 分钟）

拷回 `results_*.json`（很小，必拷）、`git push`、确认"已关机"、`COST.md` 填真实数。

完成判据：本地 `ls results_*.json | wc -l` ≥ 评测条目数（四个模型 × 各任务组，少一个就是没拷全）；`git status` 干净且 `git log -1` 是今天的提交；AutoDL 控制台该实例状态为"已关机"（截图存 `COST.md` 同目录）；`COST.md` 今天的行里**真实卡时**与**API 花费**两栏是数字而不是空或"待填"，且本周累计真实卡时 ≤4h。

今日验收：四模型结果齐 + `DEGRADATION.md` 有退化表与三条归因 + 真实卡时 ≤4h。

Day 2 • 周二：自建 200 题教学评测集 + LLM-as-judge 协议

今日总览（6h，卡时 0）：今天造的是 M4 的"主考卷"，也是你以后简历上唯一能拿出来的领域评测。三件事：**题要能区分好坏、rubric 要写死、judge 要可复现**。晚上必须完成 10 题冒烟并算清 judge 的 API 花费。

任务 1：题库骨架 + 去污染自查（约 60 分钟）

1. 定五类配额（合计 **200 题**），写进 `m4-capstone/eval/teach200_design.md`：

类别	题数	判分方式	主要看什么
口诀质量	60	LLM-as-judge 打分	是否真的助记（音/形/义挂钩），是否套话
场景例句	45	规则 + judge	3 条例句、语法正确、场景标签真实
CEFR 分级一致	35	规则（词频分位，复用周 11 的 `cefr_reward`）	输出难度是否匹配目标档
词根与事实性	35	规则 + 词典 + 人工	拆解是否成立、辨析词是否真实存在
边界与拒答	20	judge + 人工	专有名词/无意义串/中文词条时是否乱编

2. 出题：从周 7 的 prompt-only 切分里**排除已用于训练与周 11 held-out 的样本**，再做 **13-gram 交集去污染**（与训练集、与 Day 1 的评测截断集都查一遍，交集为 0 才算过）。边界题手工造（20 条，例：`iPhone`、`xjkkqzt`、`尴尬`、超长词组、A1 档生僻词）。
3. 落盘 `teach200.jsonl`：`{id, category, prompt, word, target_level, reference_points[], notes}`。`reference_points` 是这道题的**参考要点**（不是参考答案），judge 拿它对照。

完成判据：文件恰好 200 行；`category` 分布 = 60/45/35/35/20；`reference_points` **每题 ≥2 条且非空**；去污染自查打印 `CONTAMINATION HITS: 0`。

> 注：**为什么要 `reference_points` 而不是标准答案**：教学输出没有唯一正确答案，给judge一个"参考答案"会让它变成答案复读机（谁更像参考谁得分），你测的就不是质量而是相似度。给要点清单（"应指出 -tion 是名词后缀""例句应含该词的正确屈折形式"），这是 LLM-as-judge 能不能用的分水岭，面试聊评测必问。

任务 2：组装 + 参考要点标注 + 可分辨性验证（约 140 分钟）

1. 逐类写要点。60 条口诀题最费时——每条写 2-3 个要点（音/形/义中至少命中一类 + 不能是模板句），20 条边界题写"期望行为"（应指出无法拆解 / 应请求澄清 / 不应编造词根）。
2. 要点标注**双人**：你 + 一个 judge 模型先独立标一遍，你复核（分歧记 `disagree_points.csv`）。
3. 写 `judge_calibration.md`：把 20 条**故意做坏**的输出（截断、套话、编造词根、例句不含目标词、难度全部超纲）混进正常输出，先确认 rubric 打得出来（坏样本平均分应 ≤ 正常样本的 60%）。

完成判据：200 题的 `reference_points` 全部就位；`disagree_points.csv` ≥1 条也不许空（0 分歧通常说明你复制粘贴了自己的想法）；坏样本可分辨性验证通过（**20 条坏样本的 judge 均分显著低于 20 条正常样本**，两者都跑完并把两个均值写进文件）。

> 注：这一步是本周最容易被跳过、又最决定成败的一步。**judge 分不开好坏，后面所有对比表都是噪声**。花 30 分钟做坏样本，胜过事后花 3 小时解释"为什么分数看不出差异"。

任务 3：rubric 与 judge 提示词（约 80 分钟，写死不许中途改）

1. `m4-capstone/eval/rubric.md`：**四维 × 0-4 分**，每维一句可判定的描述，例：

维度	0 分	2 分	4 分
格式契约	不可解析/字段缺	可解析但 `sceneExamples` ≠3	完全合契约
事实性	词根/辨析编造	无可核验错误但无信息量	命中 ≥2 个参考要点
教学有效性	通用套话	有联想但牵强	联想具体、可用、无套话
难度匹配	与目标档差 ≥2 档	相邻档	同档

2. `judge_prompt.md`：judge 提示词原文 + **输出格式必须是 JSON** `{"format":3,"fact":2,"teach":4,"cefr":4,"reason":"..."}`。
3. 解码参数写死：`temperature=0`（**judge 不用 0.7**）、`top_p=1`、`seed=42`（若模型不支持 seed，就在 `judge_calibration.md` 里写明并只依赖 `temperature=0`）。
4. 位置偏差控制：A/B 对比时**两个方向各判一次**（swap 顺序），两次不一致记 `INCONSISTENT`。

完成判据：`rubric.md` 16 个格子（4×4）无空格；judge 脚本跑 5 条输出，**返回 5 个可解析 JSON、四个字段都是 0-4 的整数**；`INCONSISTENT` 有独立计数入口（不许静默平均掉）。

> 注：LLM-as-judge 的三个必被追问点，你都要在文件里留答：①**偏差**（位置、长度、自我偏好）→ swap 双判 + 长度差监控；②**可复现**→ `temperature=0` + 提示词原文入库 + 版本记进 `requirements-w12.txt`；③**裁判和被评模型同源**的风险 → 记录 judge 是哪个模型，并在报告里写"被评模型里没有它的同代产品"或明说这是局限。

任务 4：10 题冒烟 + 花费测算（约 50 分钟）

1. 跑通 `judge.py`（OpenAI 兼容接口）：

```
export OPENAI_BASE_URL="https://api.deepseek.com/v1"     # 或你手上的兼容端点
export JUDGE_MODEL="deepseek-chat"
python m4-capstone/eval/judge.py --inputs sample10.jsonl --out sample10.scores.jsonl
```

2. 成本测算（写进 `COST.md`，**开跑前落笔**）：正式量 = 3 档模型 × 200 题 = 600 次判定；每次输入约 1100 token（题目 + rubric + 参考要点 + 输出）、输出约 180 token；一致性抽查再加 60 样本 × 3 = 180 次。合计约 780 次调用 → 输入 ≈0.86M token、输出 ≈0.14M token。**按本仓库 `llm-router.ts` 里的参考价（deepseek-chat：输入 $0.27/M、输出 $1.1/M）估算 ≈ $0.4，即 ¥3 量级**。写上限 **¥20**（留出重跑与换更强裁判模型的余量）。

完成判据：10 条冒烟全部可解析；`COST.md` 里 Day 2 的**预估式子**完整（次数 × token × 单价，不是一句"约几块钱"）；实际账单跑完后回填真实数。

> 注：本地 Windows 上若 `import torch/pandas` 报 `OMP: Error #15`（conda 的 MKL 与 torch 自带 OpenMP 冲突），用 Week1 那个独立 venv；急就先 `set KMP_DUPLICATE_LIB_OK=TRUE`。今天 judge 脚本纯 API + CPU，真被环境问题卡住 20 分钟以上，就把脚本挪到卡上跑（**从 Day 6 的 1h 卡时额度里挪 0.3h**），别在本地耗。

任务 5：收工笔记（约 30 分钟）

NOTES ≥6 条：五类配额的取舍、一次印象最深的坏样本、分歧要点、judge 输出格式踩的坑。

今日验收：`teach200.jsonl` 恰好 200 行且去污染 0 命中 + rubric 16 格齐 + swap 双判机制就位 + 10 题冒烟可解析 + 成本式子落笔。

Day 3 • 周三：vLLM 起服务 + 明日记忆接入开关

今日总览（6.5h，卡时 2.5h）：M4 的"真实产品"这一环。今天结束要看到：`grpo-merged`（或最终交付档）被 vLLM 起成 OpenAI 兼容服务，明日记忆的 `/api/tutor` 在 `shadow` 模式下**双跑**（线上一份、影子一份）并且契约校验全过。

任务 1：算账 + 起服务前准备（约 40 分钟）

1. `COST.md`：今天 **2.5h ≈ ¥4-6**。降级：抢不到 4090 → 用任意 24G 卡（3090/4090D）；显存不够 → `--gpu-memory-utilization 0.80` 或 `--max-model-len 2048`（写进记录，因为它会影响 p95 延迟，影响明天对比表的可比性）。
2. 确认要上线的那一档：周 11 判定走 C（DPO-only）就用 `dpo-merged`，**今天不重训、不反悔**。

完成判据：`COST.md` 的 Day 3 行写好预算 + 选定档位写进 `ab_report.md` 首行（一句话说明"为什么是这一档"）。

任务 2：vLLM 起服务 + 冒烟 + 延迟基线（约 100 分钟）

```
pip install vllm -q
vllm serve /root/autodl-tmp/m4/grpo-merged \
  --served-model-name tutor-1.5b --max-model-len 4096 \
  --dtype bfloat16 --gpu-memory-utilization 0.85 --port 8000
```

本地开一条隧道把端口拉回来（AutoDL 的 SSH 端口在控制台实例页上）：

```
ssh -N -L 8000:127.0.0.1:8000 root@region-x.autodl.com -p <实例的SSH端口>
```

冒烟与压测（本地，新建 `week12/day3/probe.py`：串行 20 次 + 并发 8×30 次，记录每次端到端耗时）：

```
curl -s http://127.0.0.1:8000/v1/chat/completions -H "Content-Type: application/json" -d \
 '{"model":"tutor-1.5b","temperature":0.2,"max_tokens":512,
   "messages":[{"role":"system","content":"你是英语老师，只输出 JSON。"},
               {"role":"user","content":"为单词 serendipity 生成教学内容，目标水平 B1。"}]}'
python probe.py --base-url http://127.0.0.1:8000/v1 --model tutor-1.5b --serial 20 --concurrency 8 --n 30
```

完成判据：`curl` 返回体的 `content` 可被 `json.loads`；`probe.py` 打印 `p50/p95` 两个数字（并发 8 下）、**JSON 合法率 ≥90%（50 条里 ≥45 条）**、无 5xx。三个数记进 `latency_probe.md`。

> 注：`vllm serve` 起不来时九成是这三类（按顺序查）：① 模型目录里没有 `tokenizer_config.json`（周 11 merge 时漏拷模板文件，回头补）；② `max-model-len 4096` × 并发把 KV cache 撑爆（报错关键词 `No available memory for the cache blocks`，降 `--max-model-len 2048`）；③ vllm 与 torch 版本不匹配（**别去升 torch**，改装 `vllm` 版本或换 AutoDL 上现成的 vllm 镜像）。**这项排障总限时 60 分钟**，超时就走降级：本地 `transformers` + `TextIteratorStreamer` 起一个简陋服务，数字难看但流程能跑通，并在 `ab_report.md` 里明写"延迟数字未经 vLLM 优化，不代表部署上限"。

任务 3：接入开关（off / shadow / on）+ 单测（约 110 分钟）

1. 在 `packages/agent/src/tools/llm-router.ts` 的 `LlmRouterOptions` 里加一个显式后端选择（**只加，不改现有分支语义**），并在 `packages/agent/src/deps.ts`（或它实际的装配处）从环境变量读：
   - `TUTOR_MODEL_BACKEND=api`（默认，走 DeepSeek，线上零影响）
   - `=shadow`（先走 API 返回给用户，同时异步打一份到本地模型，只为采集对比数据）
   - `=local`（走 vLLM，用现有 `fallbackBaseUrl / fallbackApiKey / fallbackModel` 三个参数，**不改 createModel 的构造方式**）
2. **feature freeze 例外声明**：明日记忆功能开发冻结 16 周、只做服务器维护；本周这次改动属于"M4 实验平台"的授权范围，但必须做到三点——默认值是 `api`、`shadow/local` 只在本地与预发生效、单测覆盖三条分支。在 commit message 里写这句话。
3. 写单测（Vitest，跟着 `packages/agent` 现有测试的写法）：三种 env 各一条断言 + 一条"shadow 模式异常吞掉、绝不影响返回给用户的结果"。

完成判据：`pnpm -w vitest run packages/agent`（或仓库现有的测试命令）**全绿，新增 ≥4 条用例**；`TUTOR_MODEL_BACKEND` 未设置时行为与改动前**逐字节一致**（同一 mock 下断言返回体相同）。

> 注：`shadow` 模式下影子调用**必须 try/catch 吞掉一切异常且加超时**（现有 `LLM_TIMEOUT_MS = 120_000` 对影子来说太长，给影子单独设 8s）。理由直接写在 PR 注释里：本地模型第一次上真流量，超时、乱码、契约不合是**预期现象**，不能因为它们让用户看到转圈。这是产品工程与训练工程的分界线，也是你区别于"只会调脚本的候选人"的地方。

任务 4：全链路联调 + 影子双跑 30 条（约 110 分钟）

```
cd E:/CC/tomorrow-memory
pnpm -w build
TUTOR_MODEL_BACKEND=shadow pnpm --filter @tomorrow-memory/server dev
```

另开终端，把 30 个 A/B 场景逐条打进去（**同一 prompt 模板**，两侧都一样）：

```
for W in $(cut -d'"' -f4 m4-capstone/ab_scenarios_30.jsonl); do
  curl -s "http://127.0.0.1:3001/api/tutor?word=$W&level=B1" >> week12/day3/shadow_raw.jsonl
  sleep 2
done
```

（`cut -f4` 取词不一定对，**先用** `head -1 ab_scenarios_30.jsonl` **看清字段再写循环**；字段名以周 10 冻结的那份为准。跑之前把 `/api/tutor` 是否要鉴权、端口是不是 3001 确认一遍。）

完成判据：`shadow_raw.jsonl` 里 **30 条成对记录**（API 侧 + 影子侧各一份），每条含 `word, level, api_json, local_json, api_ms, local_ms, api_tokens, local_tokens, schema_ok`；`schema_ok=false` 的条数 ≤3；两侧都过 `TutorContentSchema` 的样本 ≥24。

> 注：`/api/tutor` 服务端返回的是**信封** `{word, level, content, cached}`，别拿整个信封去过 `TutorContentSchema`（要取 `content`），这是 `apps/app/src/data/queries/lookup.ts` 的 `unwrapTutorPayload` 早就处理过的形状问题——**采集脚本也要照它取**，否则你会得到 30 个假的"格式不合"。

任务 5：关机 + 记账（约 30 分钟）

隧道关掉、服务停掉、确认"已关机"、`COST.md` 回填。前端改动 `git push`（明日记忆仓库和 lab 仓库各留一份说明）。

完成判据：关机前在卡上 `nvidia-smi` 无残留 python 进程、`pkill -f "vllm serve"` 后本地 `curl http://127.0.0.1:8000/health` 连接被拒（**在卡上验，别只关隧道**——只断隧道时本地也会"连不上"，那是假信号）；AutoDL 控制台状态"已关机"（截图存档，**不是销毁**）；`COST.md` 今天一行写满**真实卡时（≤2.5h）**与累计值；两个仓库 `git status` 均为 clean，且明日记忆仓库的 commit message 里点名本次改动的文件（`llm-router.ts` / `tutor.ts` / 新增单测）。

今日验收：`p50/p95` 有数 + JSON 合法率 ≥90% + 单测全绿（含默认行为不变）+ 影子双跑 30 条成对。

Day 4 • 周四：30 场景 A/B 收口 + 成本与延迟对比表

今日总览（6h，卡时 ≤1h）：把昨天的原始记录变成一张能被追问的表。**这张表是面试叙事的落点**（"小模型替换 API 降本"有没有资格说这句话，全看今天）。

任务 1：A/B 统计（约 150 分钟）

1. 写 `ab_report.md`，三张子表：
   - **契约与质量**：JSON 合法率、`sceneExamples=3` 命中率、周 11 三个奖励函数当"自动质量代理"跑一遍（`cefr/fact/format` 三项分）；judge 分数（Day 2 的 rubric，`temperature=0` + swap 双判）。
   - **延迟**：两侧 `p50/p95/max`，两侧**同一台机器同一并发**；标注 `max-model-len` 与并发设置。
   - **成本**：API 侧按真实 token × `llm-router.ts` 里的单价；自建侧按"卡时价 ÷ 该并发下每小时可完成请求数"。

2. 成本换算式必须写出来，例：4090 ¥2.0/时 → 每分钟约 ¥0.033；若并发 8 下每小时完成 N 条，则单条 ≈ `0.033×60/N` 元。用昨天的实测 N，不要拿别人的吞吐数字。

完成判据：三张子表齐，**每个数字旁边有出处**（文件名或脚本名）；成本表给出"单条请求 ¥x vs ¥y"两个数与倍数；并写出**未计入项**（运维、模型迭代、失败重试、GPU 空转）——这一条不写，成本对比就是不诚实的。

> 注：延迟大概率是 API 侧更快（DeepSeek 是大集群推理，你这台按量 4090 只有你一个人在压）。**这不是失败**，是结论。真实降本逻辑通常不在单次延迟，而在：可离线批量生成 + 无按 token 计费 + 私有化可控 + 高峰期不被限流。把这四条里你能验证的写进表；不能验证的，写成"待验证假设"。把不利数字讲清楚，比编一个好看的数字值钱十倍。

任务 2：把 A/B 数字接回评测集（约 100 分钟）

1. 跑 `teach200.jsonl` 的正式评测：三档模型 × 200 题 → judge 打分 → `teach200_scores.csv`。
2. 一致性抽查：`random.seed(42)` 抽 **60 题**，同一样本**重跑 3 次**（`temperature=0` 下仍可能因服务端实现而不完全一致——这正是要查的），逐次对比。

完成判据：200 题 × 3 档 = 600 条分数齐、无缺失；一致性抽查给出**"三次分差均 ≤1 的样本占比"，目标 ≥90%**；不达标的样本单独列 `judge_unstable.csv` 并给出你的处理（这些题要么改 rubric 要么删，**不许偷偷删**）。

> 注：真实 API 花费今天到顶也就 ¥3-8（Day 2 的式子 ×1.2 的余量）。**上限 ¥20**：超了说明你在反复重跑——把 judge 结果缓存到 `scores_cache.jsonl`（以 `model+id+prompt_hash` 为键），第二次永远命中缓存。这个缓存本身就该在 repo 里，它是"可复现"的一部分。

任务 3：失败请求归档 + 主观分（约 80 分钟）

1. 把 30 条里 `schema_ok=false` 或质量明显差的样本存 `m4-capstone/ab/failures/`（含完整输入输出与你的原因判断），并**回写进周 11 的** `FAILURES.md`（对齐阶段与服务/部署阶段的失败合并成一份）。
2. 主观分（周 10 冻结协议里那一列）：你自己 + 找一个能看的同学（牛客/即刻拉一个即可，不必现场找人），各按 0-2 分给 10 条打分，**分歧 ≥2 分的重看一次**。

完成判据：失败归档 ≥3 条且每条有"原因判断"；主观分两人各 ≥10 条、给出平均与分差；`FAILURES.md` 条目总数比周 11 增加 ≥2。

> 注：**这份 `FAILURES.md` 是周 14「失败案例复盘」的主素材，本周不合并就再也拼不回来**（服务侧报错的上下文在终端 scrollback 里，明天就没了）。周 14 面试最值钱的部分是"现象→定位→修复"，而现象必须带原始报错行。

任务 4：收工笔记（约 30 分钟）

NOTES ≥5 条：一个不利数字怎么解释、成本换算里最脆弱的假设、judge 不稳定的题长什么样。

今日验收：三张子表 + 600 条分数 + 一致性 ≥90%（或不达标已列清单）+ 失败归档进 `FAILURES.md`。

Day 5 • 周五：模型上传 HF/ModelScope + 可复现 repo 收口

今日总览（6.5h，卡时 ≤1.5h）：交付物从"我有"变成"别人能下、能跑"。今天是**诚信日**：model card 里哪些是复现、哪些是自写，必须标清楚。

任务 1：上传前自查（约 50 分钟）

1. 三项硬自查，逐条截图或粘贴输出进 `m4-capstone/RELEASE_CHECKLIST.md`：
   - **许可**：基座是 Qwen2.5（Apache-2.0，仓库里已带 `LICENSE`，一并上传并保留 `NOTICE`）；你的 adapter/merged 权重衍生自它，**model card 的 license 字段不能写成别的**。
   - **数据合规**：数据集只含周 7 已去标识的合成/公开数据；`grep -ri` 查一遍 repo 里有没有 `api_key`、`sk-`、真实邮箱/手机号、原始日志片段。**训练脚本里写死的 key 现在必须换成环境变量**。
   - **去污染声明**：model card 里明写"评测集 `teach200.jsonl` 未参与任何训练，13-gram 交集 0 命中"。
2. `du -sh` 看一下权重体积，决定是否需要 `git lfs`/直接走 Hub 上传。

完成判据：三条各有勾选与证据；`grep -rn "sk-\|api_key\|Bearer " m4-capstone/` **0 命中**（除了文档里的占位符 `<你的API Key>`）。

> 注：如果 repo 历史上**曾经**提交过明文 key，光删文件没用——历史里还在。今天就处理掉：换 key（最省事，也是正确做法）+ 在 README 注明"历史提交含已作废的密钥"。**别不当回事**：租卡账号里的余额就是你的损失上限。

任务 2：上传 Hugging Face（约 120 分钟）

```
pip install -U "huggingface_hub[cli]" -q
export HF_ENDPOINT=https://hf-mirror.com
huggingface-cli login                    # 粘贴 Read token（在 HF 网站 Settings → Access Tokens 建）
hf upload <你的用户名>/mingtianjy-tutor-1.5b /root/autodl-tmp/m4/grpo-merged --repo-type model
```

`README.md`（即 model card）必须有 front matter 与这些段落：模型简介 / 基座 / 训练三阶段各自的数据与超参 / **评测结果表**（直接引 Day 1 与 Day 4 的数字，标注 `--limit 300` 截断）/ 使用方式（`apply_chat_template` 的正确调用 + vLLM 启动命令 + `max-model-len` 建议）/ 已知局限（Day 1 的退化、Day 4 的失败样本类型）/ **差异声明**。

完成判据：匿名状态下 `git clone https://hf.co/<用户名>/mingtianjy-tutor-1.5b`（或 `huggingface_hub.hf_hub_download`）**能拉到权重并能被** `AutoModelForCausalLM.from_pretrained` **加载**；model card 六段齐。

> 注：hf-mirror 对**下载**很稳，对**上传**不一定通。上传这一步要在网络条件好的时段做（或在卡上做，卡上一般更快），超时不要反复重试撞 Hub 的限流；**万一 HF 彻底不行，就把 ModelScope 当作主链接**（国内直连可用），HF 留到周 13 周末再补，别让它挡住 Day 5 的主线。

任务 3：上传 ModelScope（约 70 分钟）

```
pip install modelscope -q
modelscope login --token <你的魔搭 Token>
modelscope upload <你的用户名>/mingtianjy_tutor_1_5b /root/autodl-tmp/m4/tutor-1.5b --repo-type model
```

页面里的模型介绍与 HF 一致（复制过来，两边版本号/日期一致，**不一致会被面试官当成"两份材料两个说法"**）。

完成判据：魔搭模型页可匿名访问、文件列表含权重与 tokenizer；与 HF 的 commit 哈希或日期对得上。

任务 4：可复现 repo 收口（从零跑通验收）（约 100 分钟）

`llm-posttraining-lab/m4-capstone/` 的最终结构（缺哪个补哪个）：

目录/文件	内容
`README.md`	项目一页纸：动机 → 三阶段 → 指标 → 局限 → 复现步骤
`repro.sh`	一键：下数据 → SFT → DPO → 奖励 → GRPO → 评测 → 起服务
`requirements-w11.txt` / `requirements-w12.txt`	锁版本（含 `trl`、`vllm`、`lm-eval`）
`sft/` `align/` `eval/` `ab/`	各阶段脚本 + 配置 + 日志 + 曲线图
`align/FAILURES.md`	合并后的失败清单（周 11 + 周 12）
`NOTICE-DIFFERENCES.md`	哪些代码来自 LLaMA-Factory / TRL example / 周 8-9 自己的脚本，哪些本周新写

验收动作（不是自查，是真跑）：换一台机器（本地另开目录或卡上新目录），只 `git clone` + `bash repro.sh --stage eval --smoke`（你自己实现的 `--smoke`：20 题、10 step 的最小通路）。

完成判据：`--smoke` 在**新目录里从 clone 到出结果一次跑通**，中途不需要你手工修任何路径；把终端记录（去掉 key）贴进 README 的 "Reproduction log" 一节。**跑不通就是没收口**，今天别做别的。

> 注：`repro.sh` 里凡是要花钱的环节（API 标注、租卡训练）都要有 `--dry-run` 与显式提示"这一步预计 ¥x"。一个能白嫖跑通评测、要花钱才跑训练的 repo，比一个什么都要卡的 repo 友好十倍，也诚实十倍。

任务 5：README 与差异声明定稿（约 50 分钟）

把"复现 / 自写 / 借用"三类逐条列出，一句话说明每类的作用。**面试诚信不是装饰**：被追问时说不清哪段是自己写的，比承认"这里我参考了 TRL 的 example"糟糕得多。

完成判据：`NOTICE-DIFFERENCES.md` ≥12 条，覆盖训练、评测、服务、数据四条线；README 首屏能看懂这个项目在做什么（找一位没跟过你 12 周的同学看 60 秒，让他复述一遍，复述不出来就重写开头 200 字）。

今日验收：两个模型链接可匿名拉取 + `--smoke` 从零跑通 + 三项硬自查有证据 + 差异声明 ≥12 条。

Day 6 • 周六：M4 系列博客第 3 篇 + 面试叙事 200 字定稿 + 讲稿

今日总览（6h，卡时 ≤1h）：把 M4 说成三样东西：一篇公开技术文（细节）、一段 200 字（简历与自我介绍）、一份 15 分钟讲稿（项目面）。**三段口径必须完全一致**，不一致的地方面试官一定会抓。

任务 1：博客第 3 篇成文（约 120 分钟）

1. 标题（三选一或自拟）：《我把自己的 App 换成了自己训的 1.5B 模型：成本、延迟和一地失败》《200 题评测集是怎么把 judge 逼疯的》《从 API 调用方到模型供应方：明日记忆换心脏记》。
2. 结构（**2000-2600 字**，M4 系列最后一篇，配额到此为止）：
   - ① 为什么要在真实产品里测（离线分数骗人的两个真实例子）；
   - ② 评测集设计：五类配额、`reference_points` 而不是参考答案、坏样本可分辨性验证（贴 rubric 表）；
   - ③ vLLM + 开关式灰度：默认 `api`、`shadow` 双跑、影子 8s 超时（贴一段真实代码）；
   - ④ 结果三张表（质量 / 延迟 / 成本），**把不利数字放在前面并解释**；
   - ⑤ 已知局限与下一步（replay 混数据、7B 或更大模型、蒸馏到更小模型）。

完成判据：≥2000 字、图/表 ≥4、**至少一处主动写出对我不利的结论**、所有数字与 `DEGRADATION.md` / `ab_report.md` 一致（写一句"数据出处：<文件路径>"）。

任务 2：发布（约 40 分钟）

发《从全栈工程师到后训练》；三篇 M4 博客链接互指；`m4-capstone/README.md` 顶部加博客链接。

完成判据：无痕窗口能打开 + README 里三个链接（周 10/11/12 三篇）都有效。

任务 3：面试叙事 200 字定稿（约 80 分钟）

结构固定五句（顺序不许换，这是总计划定的链路）：

```
① 真实产品：明日记忆（我的全栈 App，有真实用户与查词日志）
② 日志构造数据：去标识 + self/evol-instruct 合成 <N> 条指令数据并发布（<数据集名>）
③ 三阶段训练：Qwen2.5-1.5B 上 LoRA SFT → 偏好对 DPO（<对数>对）→ 可验证奖励 GRPO（<三个奖励名>）
④ 评测：通用基准（带 base 对照，退化 <x> pt）+ 自建 200 题教学评测集（LLM-as-judge，rubric + 一致性 <y>%）
⑤ 线上：vLLM 起服务 + 开关式灰度 A/B，30 个真实场景，成本 <¥a> vs <¥b>，延迟 <p50> vs <p50>
```

完成判据：成文 **200-240 字**（多了就删形容词，不许删数字）；旁边放一张 `叙事溯源表.md`：**每个数字一行 → 出自哪个文件哪一列**；表里若有任何一个数字找不到出处，这个任务就是没完成。

> 注：这一段是你整场面试的开场白，也是简历"项目经历"第一段的底稿。写的时候用嘴念一遍：**念到任何一处需要想"这个词怎么读"或"这句是什么意思"，就是写坏了**，改到能一口气念完为止。

任务 4：15 分钟讲述版讲稿（约 50 分钟）

`m4-capstone/TALK-15min.md`：动机 2min → 方案与数据 3min → 训练 4min → 评测与 A/B 4min → 失败与改进 2min。每段标"这里可能被追问什么"（各 ≥2 条），**把周 11/12 的 `FAILURES.md` 里最硬的两条埋进"失败与改进"段**。

完成判据：计时对着讲一遍 **13-15 分钟**（不超时、不少于 12 分钟）；每条追问旁白有 ≥3 行应答要点。

任务 5：周复盘 + gaps（约 70 分钟）

`review.md` 五块 + **Phase 3 三周（10/11/12）合并复盘**（150 字：这一整块最缺的能力是什么）。gaps 明确分派：技术洞 → 周 14/15 的对应条目；求职动作 → 周 13。

完成判据：五块齐 + 三周合并复盘段写完 + gaps 每条有"哪天 + 什么动作"。

今日验收：博客链接可访问 + 200 字叙事（200-240 字）与溯源表齐 + 讲稿计时通过 + 复盘五块。

Day 7 • 周日：补漏 + 闭卷复测 + Phase 3 总验收 + 周 13 预习

今日总览（5h）：今天不写新代码。**这是整个技术能力部分（周 1-12）的终点验收**，判据比前 11 周硬：能跑的必须现场跑给你看，跑不出来就是没通过。

任务 1：补漏（约 80 分钟）

打开 `gaps.md`。**规则：最多补 2 个洞，贪多必失**。优先级：Day 5 的 `--smoke` 复现（对外交付的门面）→ Day 4 的 judge 一致性（所有质量数字的地基）→ 其余。

完成判据：这 2 条从 `[ ]` 变 `[x]`，并写一句"怎么补的 + 证据在哪"。

任务 2：闭卷复测（约 45 分钟，别跳）

不看任何资料，写完四题： 1. 为什么 LLM-as-judge 必须固定 `temperature` 并做重跑一致性抽查？不固定会具体坏在哪一步？ 2. 用 3-5 行伪代码写出 GRPO 的组内优势计算（含 G 个采样、均值、标准差、KL 项在损失里的位置）。 3. DPO 与 GRPO 各自需不需要在线采样？各自需要几个模型同时驻留显存？为什么？ 4. 写清你本周三个奖励函数的名字与组合方式，并回答：**为什么"中文释义是否准确"不进 reward 而进评测**？

完成判据：**四题全对**（第 4 题是本周的智力产出，答不到"可验证 vs 需要世界知识"这条界上就是没懂）。错的重做一遍，并写进周 15「项目深挖」的必练清单。

任务 3：Phase 3 总验收（约 90 分钟，逐项现场跑）

#	验收项	判据（现场跑，不接受"我记得没问题"）
1	三档模型	`sft-merged` / `dpo-merged` / `grpo-merged`（或降级说明）在持久盘 + 已 push 的配置可加载；`smoke_template.py` 三跑三通过

#	验收项	判据（现场跑，不接受"我记得没问题"）
2	通用基准	`eval/` 四模型结果 JSON 齐； `DEGRADATION.md` 有最大退化值与三条归因；四个模型同 `--limit`
3	自建评测集	`teach200.jsonl` 恰好 200 行、配额 60/45/35/35/20、去污染 0 命中、坏样本可分辨性有数
4	judge 可复现	`rubric.md` 16 格齐； `temperature=0` 与 swap 双判在脚本里可见；一致性 ≥90%（或不稳定清单已列）
5	真实接入	起一次 `vllm serve` + `curl` 返回可解析 JSON； `TUTOR_MODEL_BACKEND` 三态单测全绿、默认行为不变
6	A/B 报告	质量/延迟/成本三张子表，每数有出处，未计入项已写明
7	公开交付	HF 与魔搭链接**在另一台机器上匿名拉取并加载成功**； `--smoke` 一次跑通；三篇博客可访问
8	失败资产	`FAILURES.md` ≥10 条（周 11+12 合并），每条四段式且含原始报错行
9	叙事一致	200 字叙事、博客、README 三处数字**完全一致**（抽查 5 个数字逐个对文件）

完成判据（通过标准，比前 11 周硬）：**9 项里 ≥8 项通过，且第 1、第 5、第 7 项为强制项——任一强制项不过即本周不合格**，必须占用周 13 Day 1 的前 3 小时补齐（周 13 的刷题计划相应顺延，**不许压缩简历与投递动作来抵**）。判定的物理证据：把上表每行的命令现场跑一遍，输出重定向进 `week12/accept.log` （**带时间戳，一张表 9 段，不许只贴成功的那几段**），验收表格里每格填 `通过/未通过 + 该日志的行号` 。

> 注：第 5、7 项设成强制，是因为它们恰好是"科班实习生讲不出来的那半条链路"（真实产品接自己的模型 + 别人能下载能跑）。模型分数差两个点不影响你能不能拿到面试，这两件事没做才影响。

任务 4：周 13 预习（约 40 分钟）

周 13 起，**性质完全变了：不再训模型，开始为面试服务**。今晚只做三件事： 1. 建 `E:/study/posttrain/jobhunt/` ，里面空建四个文件： `leetcode.md` 、 `ml-coding.md` 、 `bagu.md` 、 `applications.csv` （表头先按周 13 的九列写死： `公司,档位,岗位名,渠道,内推人/内推码,投递日,当前状态,最后动作日,复盘链接` ，字段内不许出现逗号）。 2. 打开 LeetCode 题库里的「热题 100」，只看前 20 题的标题，把"一眼就知道怎么写"的题号抄下来（明天开始它们不再重复排进新题配额）。 3. 在 `review.md` 末尾写一行：**未来 4 周我的技术资产不会再增长，只会贬值——除非我把它讲出来**。

完成判据：四个文件建好且 `applications.csv` 有表头；"一眼会"的题号清单 ≥5 个；那一行写完。

任务 5：缓冲与移交（约 45 分钟）

把没闭环的东西打包给 Phase 4，格式"缺什么 → 现在在哪个文件 → 排到周 13/14/15 哪天"。典型三类：Day 1 发现要加 replay 重跑（**排到周 13 的 4h 卡时额度里**，别顺手开 10h）、judge 不稳定的题目要改 rubric（周 14 评测八股的实战素材）、ModelScope 或 HF 少一个（周 13 Day 1 补，**必须在第一批投递之前**）。

完成判据： `gaps.md` 新增一节 `## 交给 Phase 4` ，≥3 条且三字段齐。

今日验收：总验收 ≥8 项且三项强制项全过 + 闭卷四题全对（或已重做） + 预习与移交齐。

三、常见问题与纪律

- **某天没完成怎么办？** 按"天-任务号"记进 `gaps.md`，Day 6 与 Day 7 就是缓冲区。**本周优先级排序**：Day 3 的真实接入 > Day 4 的对比表 > Day 5 的可复现收口 > Day 2 的评测集 > Day 1 的基准。排序靠后的可以带瑕疵前进，靠前的不行。
- **"数字不好看能不能不写"**：不能，**但可以解释**。本周所有不利数字（延迟更高、judge 不稳、退化 x pt）都要写进博客与 README，并附"我的处置"。面试官不怕你项目有毛病，怕你看不见毛病——看不见毛病的人，入职后就是那个把线上事故藏到周五下午的人。
- **卡时与花费纪律**：本周上限 10h / ≤¥30。评测是**一次性大批量**动作，开卡前把命令写全、`tee` 到日志、断网也不丢结果；跑完立刻关机（**不是销毁**）。API 花费上限 ¥20，靠 `scores_cache.jsonl` 缓存把重跑成本压到零。
- **别在这周碰训练**。想再调一次 DPO、想加一轮 GRPO——那是周 11 的债，写进 gaps 交周 13 的 4h 额度。本周把"读数"做可信，比再多 0.5 个点有用得多。
- **产品侧改动的边界**：`TUTOR_MODEL_BACKEND` 的默认值必须是 `api`；任何让线上路径依赖本地模型的改动都不许合。功能冻结还在生效，本周这次是唯一的例外，理由已写进 commit message。
- **AI 工具的使用边界**：允许——查 `lm-eval` / `vllm` 的参数、解释报错、review 你的 rubric 是否有歧义。不允许——替你写评测集要点（那是你判断"什么是好教学输出"的能力，面试直接考）、替你写 200 字叙事（写完不像你说的话，一开口就穿）。
- **收尾一句话**：周 1-12 到此为止。你手上现在应该有的东西是：一个能下载的模型、一个能跑的 repo、一份带 base 对照的评测表、一份 10+ 条的失败清单、三篇博客、一段 200 字。周 13 起，任务只剩一件——**把这些东西说清楚，并且能下场写代码**。